

Strings in Java

I. Introduction to Strings: Working with Text in Java

When we write programs, numbers alone are not enough. Real applications deal with **names, messages, passwords, emails, and sentences** — all forms of text. This is where **Strings** become essential.

In Java, a **String** represents a sequence of characters enclosed within double quotation marks.

```
String name = "Anum";
```

At first glance, this looks simple. But here is the interesting part: a String in Java is **not a primitive data type**. It is actually an **object** belonging to the `String` class.

Think of a String as a carefully sealed envelope containing characters. Once sealed, we cannot directly change its contents — a concept that leads us to one of Java's most important ideas: **immutability**.

Why does Java treat text this way? Because text data must remain secure, predictable, and efficient across programs.

II. Creating Strings in Java

Java allows us to create Strings in multiple ways. Each method affects memory and performance differently.

1. Using String Literal

The most common approach:

```
String city = "Karachi";
```

Here, Java stores the String inside a special memory area called the **String Pool**. If another variable stores `"Karachi"`, Java reuses the same object instead of creating a new one. This saves memory — smart, right?

2. Using the `new` Keyword

We can also explicitly create a String object:

```
String city = new String("Karachi");
```

This forces Java to create a new object in memory, even if the same value already exists.

So we ask ourselves: when should we use this? Usually, string literals are preferred because they are more memory-efficient.

3. Empty Strings

Sometimes we create placeholders:

```
String message = "";
```

This acts like an empty notebook waiting for content.

III. String Immutability: Why Strings Cannot Change

One of Java's most fascinating features is that **Strings are immutable**.

What does immutable mean?

It means once a String object is created, its value cannot be modified.

Consider this:

```
String text = "Hello";  
text = text + " World";
```

Did we change the original String? No. Java created a **new object** containing `"Hello World"`.

Why design Strings this way?

- Security (important for passwords and file paths)
- Thread safety
- Efficient memory sharing through the String Pool

Imagine writing with permanent ink instead of pencil — you cannot erase; you create a new page instead.

IV. Common String Methods in Java

The true power of Strings appears through built-in methods. Java provides many tools for manipulating text easily.

1. Length of a String

```
String name = "Java";
System.out.println(name.length());
```

Returns the number of characters.

2. Converting Case

```
name.toUpperCase();
name.toLowerCase();
```

Useful when comparing user input regardless of capitalization.

3. Character Access

```
name.charAt(1);
```

Returns the character at index 1.

Remember — indexing starts from 0.

4. Comparing Strings

Using `==` compares memory references, not content. Instead, we use:

```
name.equals("Java");
```

Or ignoring case:

```
name.equalsIgnoreCase("java");
```

Think of it like comparing meanings rather than addresses.

5. Substring Extraction

```
String text = "Programming";
text.substring(0, 6);
```

This extracts "Progra".

It is like cutting a slice from a cake without changing the original cake.

6. Searching Inside Strings

```
text.indexOf("gram");
```

Returns the starting position of the word.

V. String Concatenation and Joining Text

Combining Strings is extremely common in programming.

Using the + Operator

```
String first = "Hello";
String second = "World";
String result = first + " " + second;
```

Output: Hello World

Simple and readable.

Using `concat()` Method

```
first.concat(second);
```

Both approaches create a new String object.

Performance Consideration

Repeated concatenation inside loops can slow programs because many temporary objects are created. For heavy text operations, Java provides **StringBuilder**, which we will briefly explore later.

VI. StringBuilder and StringBuffer

If Strings cannot change, how do we efficiently modify text repeatedly?
Java introduces mutable alternatives.

1. StringBuilder

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" World");
```

Unlike String, this modifies the same object.
Advantages:

- Faster performance
- Less memory usage
- Ideal for loops and dynamic text generation

2. StringBuffer

Similar to StringBuilder but **thread-safe**.
Difference:

Feature	StringBuilder	StringBuffer
Speed	Faster	Slightly slower
Thread Safety	No	Yes

Most modern applications prefer StringBuilder unless synchronization is required.

VII. String Memory Concept: The String Pool

Java optimizes memory using a special area called the **String Constant Pool**.
Example:

```
String a = "Java";
String b = "Java";
```

Both variables reference the same object.
But:

```
String c = new String("Java");
```

This creates a new object outside the pool.
Why does this matter? Because memory efficiency becomes critical in large applications handling thousands of text values.

VIII. Escape Sequences in Strings

Sometimes we need special formatting characters.

Escape Sequence	Meaning
\n	New line
\t	Tab space
\"	Double quote
\\	Backslash

Example:

```
System.out.println("Hello\nWorld");
```

Output appears on two lines.
Escape sequences act like invisible instructions guiding how text should appear.

IX. Common String Operations in Programs

Let us examine real-world uses.

1. Reversing a String

```
String str = "Java";
for(int i = str.length()-1; i >= 0; i--) {
    System.out.print(str.charAt(i));
}
```

2. Checking Palindrome

```
String original = "madam";
String reverse = "";

for(int i = original.length()-1; i >= 0; i--) {
    reverse += original.charAt(i);
}

if(original.equals(reverse)) {
    System.out.println("Palindrome");
}
```

3. Removing Spaces

```
text.replace(" ", "");
```

These examples show how Strings support logical problem-solving.

X. Best Practices When Working with Strings

Good programmers treat Strings carefully.

We should:

- Use `.equals()` instead of `==`
- Prefer `StringBuilder` for repeated modifications
- Avoid unnecessary object creation
- Validate user input properly
- Keep Strings readable and meaningful

Small habits create efficient and maintainable programs.

Conclusion

Strings are the voice of a program — they allow software to communicate with humans through words rather than numbers. In this lecture, we explored how Strings are created, stored, compared, and manipulated in Java. We examined immutability, String methods, concatenation, memory optimization through the String Pool, and advanced tools like `StringBuilder`.

If arrays organize data, Strings give it meaning. They transform raw logic into readable communication, enabling applications to display messages, process input, and interact naturally with users. Mastering Strings equips us with the ability to handle real-world information effectively, making our Java programs not only functional but expressive and intelligent.